



Aula 4 — Arrays e Funções em C

Até aqui, já aprendemos bastante coisa.

Já conseguimos criar variáveis, fazer cálculos, tomar decisões e repetir tarefas usando `for` e `while`.

Mas começa a aparecer um problema.

Imagine que queremos guardar as notas de 5 alunos.

Poderíamos fazer assim:

```
float nota1;  
float nota2;  
float nota3;  
float nota4;  
float nota5;
```

Funciona.

Mas imagine agora que temos 100 alunos.

Teríamos que criar 100 variáveis?

Não seria muito divertido.

É aí que entram os **arrays**.

Depois vamos conhecer as **funções**, que vão nos ajudar a organizar o nosso código e evitar que tenhamos de escrever a mesma coisa várias vezes.

1. O que é um array?

Um array é uma forma de guardar vários valores do mesmo tipo dentro de uma única variável.

Pensa em um armário com várias gavetas.

Em vez de ter:

```
nota1  
nota2  
nota3
```



nota4
nota5

podemos ter:

```
notas
├─ 0
├─ 1
├─ 2
├─ 3
└─ 4
```

Cada posição guarda uma informação.

Em C, podemos criar um array assim:

```
float notas[5];
```

Isso significa:

“Crie um array chamado `notas` capaz de guardar 5 valores do tipo `float`.”

2. Uma coisa importante: a contagem começa no zero

Aqui temos uma pequena pegadinha do C.

Quando criamos:

```
float notas[5];
```

temos 5 posições, mas elas são numeradas assim:

```
Posição 0
Posição 1
Posição 2
Posição 3
Posição 4
```

Não existe a posição 5.

Então:

```
notas[0]
```

é a primeira posição.

```
notas[1]
```

é a segunda.

E assim por diante.

No começo pode parecer estranho.

Mas depois de usar algumas vezes, você começa a pensar naturalmente em 0, 1, 2, 3... .

3. Colocando valores dentro de um array

Podemos colocar valores diretamente:

```
int numeros[5] = {10, 20, 30, 40, 50};
```

Agora temos:

```
numeros[0] → 10  
numeros[1] → 20  
numeros[2] → 30  
numeros[3] → 40  
numeros[4] → 50
```

Podemos acessar uma posição específica:

```
printf("%d", numeros[0]);
```

Resultado:

```
10
```

Ou:

```
printf("%d", numeros[3]);
```

Resultado:

40

4. Arrays e for: uma combinação muito importante

Agora vem uma das partes mais interessantes.

Lembra do `for` da aula anterior?

Ele combina muito bem com arrays.

Por exemplo:

```
#include <stdio.h>

int main() {

    int numeros[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i < 5; i++) {
        printf("%d\n", numeros[i]);
    }

    return 0;
}
```

Resultado:

```
10
20
30
40
50
```

O `for` vai passando por cada posição do array.

Primeiro:

```
numeros[0]
```

Depois:

```
numeros[1]
```

Depois:

```
numeros[2]
```

E continua até:

```
numeros[4]
```

Isso é extremamente útil quando temos muitos dados.

5. Exemplo real — notas de um aluno

Vamos fazer algo mais interessante.

Imagine que queremos guardar 5 notas e depois calcular a média.

```
#include <stdio.h>

int main() {

    float notas[5];
    float soma = 0;
    float media;

    for (int i = 0; i < 5; i++) {

        printf("Digite a nota %d: ", i + 1);
        scanf("%f", &notas[i]);

        soma = soma + notas[i];
    }

    media = soma / 5;

    printf("\nMedia final: %.2f\n", media);

    return 0;
}
```

Aqui estamos fazendo várias coisas.

O array:

```
float notas[5];
```



guarda as notas.

O `for` percorre as posições.

O `scanf()` recebe cada nota.

E:

```
soma = soma + notas[i];
```

vai acumulando os valores.

No final:

```
media = soma / 5;
```

calcula a média.

Já estamos começando a criar programas que conseguem trabalhar com vários dados.

Parte 2 — Funções

6. O problema de repetir código

Agora imagine que temos um programa com vários cálculos.

Por exemplo:

```
printf("Bem-vindo ao sistema!\n");  
printf("Bem-vindo ao sistema!\n");  
printf("Bem-vindo ao sistema!\n");
```

Estamos repetindo código.

E isso pode ficar muito pior em programas grandes.

Imagine ter 500 linhas e precisar repetir o mesmo processo em vários lugares.

Seria uma confusão.

As funções existem justamente para ajudar nisso.

7. O que é uma função?

Uma função é um bloco de código criado para realizar uma determinada tarefa.

Podemos pensar nela como uma pequena máquina.

Você coloca alguma coisa nela:

```
ENTRADA
  ↓
[ FUNÇÃO ]
  ↓
RESULTADO
```

Por exemplo, podemos criar uma função para mostrar uma mensagem:

```
void mensagem() {
    printf("Bem-vindo ao programa!\n");
}
```

Agora podemos chamar essa função:

```
mensagem();
```

8. Criando nossa primeira função

Vamos fazer um programa completo:

```
#include <stdio.h>

void mensagem() {
    printf("Bem-vindo ao nosso programa!\n");
}

int main() {
    mensagem();

    return 0;
}
```

Quando o programa chegar em:

```
mensagem();
```

ele vai executar o código que colocamos dentro da função.

Resultado:

```
Bem-vindo ao nosso programa!
```

A vantagem é que podemos chamar a função várias vezes:

```
mensagem();  
mensagem();  
mensagem();
```

E não precisamos escrever o `printf()` três vezes.

9. Funções com parâmetros

Agora podemos deixar as funções ainda mais interessantes.

Imagine que queremos uma função que cumprimente uma pessoa.

Podemos passar o nome para ela.

```
#include <stdio.h>  
  
void cumprimentar(char nome[]) {  
    printf("Ola, %s!\n", nome);  
}  
  
int main() {  
    cumprimentar("Jose");  
    cumprimentar("Carlos");  
    cumprimentar("Maria");  
  
    return 0;  
}
```

Resultado:


```
Ola, Jose!  
Ola, Carlos!  
Ola, Maria!
```

O valor que colocamos dentro dos parênteses é chamado de **argumento**.

E o `nome` dentro da função é o **parâmetro** que recebe esse valor.

Não precisa complicar isso agora.

Pensa simplesmente:

“Eu mando uma informação para a função e ela usa essa informação.”

10. Funções que retornam valores

Até agora nossas funções apenas executaram alguma coisa.

Mas uma função também pode fazer um cálculo e devolver um resultado.

Por exemplo:

```
int somar(int a, int b) {  
  
    return a + b;  
  
}
```

Essa função recebe dois números e devolve a soma.

Podemos utilizá-la assim:

```
#include <stdio.h>  
  
int somar(int a, int b) {  
  
    return a + b;  
  
}  
  
int main() {  
  
    int resultado;  
  
    resultado = somar(10, 20);
```

```
    printf("Resultado: %d\n", resultado);

    return 0;
}
```

Resultado:

Resultado: 30

Aqui aconteceu o seguinte:

```
10 e 20
  ↓
somar()
  ↓
30
```

A função recebeu dois valores, fez o cálculo e devolveu o resultado.

11. void e int

Você pode ter percebido uma diferença.

Antes usamos:

```
void mensagem()
```

Agora usamos:

```
int somar()
```

Isso acontece porque as duas funções têm comportamentos diferentes.

`void` significa que a função não retorna um valor.

```
void mensagem() {
    printf("Ola!");
}
```

Já `int` significa que a função vai retornar um número inteiro.

```
int somar(int a, int b) {  
    return a + b;  
}
```

Existem outros tipos de retorno, como:

```
float  
double  
char
```

Mas vamos aprender isso aos poucos.

12. Arrays e funções juntos

Agora vamos juntar os dois conceitos.

Podemos criar uma função para calcular a média de um array de notas:

```
#include <stdio.h>  
  
float calcularMedia(float notas[], int tamanho) {  
  
    float soma = 0;  
  
    for (int i = 0; i < tamanho; i++) {  
        soma = soma + notas[i];  
    }  
  
    return soma / tamanho;  
}  
  
int main() {  
  
    float notas[5] = {12, 15, 10, 14, 16};  
  
    float media = calcularMedia(notas, 5);  
  
    printf("Media: %.2f\n", media);  
  
    return 0;  
}
```

Resultado:

```
Media: 13.40
```

Agora temos um programa muito mais organizado.

O `main()` não precisa saber todos os detalhes de como a média é calculada.

Ele simplesmente diz:

```
calcularMedia(notas, 5);
```

E a função faz o trabalho.

Essa é uma das grandes ideias da programação:

dividir um problema grande em partes menores.

13. Um pequeno projeto — Sistema de notas

Vamos terminar a aula com um pequeno projeto.

O programa vai:

- Receber 5 notas;
- Guardar as notas em um array;
- Calcular a média usando uma função;
- Informar se o aluno foi aprovado ou reprovado.

```
#include <stdio.h>

float calcularMedia(float notas[], int tamanho) {

    float soma = 0;

    for (int i = 0; i < tamanho; i++) {
        soma += notas[i];
    }

    return soma / tamanho;
}

int main() {

    float notas[5];

    printf("=== SISTEMA DE NOTAS ===\n\n");
```

```

for (int i = 0; i < 5; i++) {

    printf("Digite a nota %d: ", i + 1);
    scanf("%f", &notas[i]);
}

float media = calcularMedia(notas, 5);

printf("\nMedia final: %.2f\n", media);

if (media >= 10) {
    printf("Resultado: Aprovado!\n");
} else {
    printf("Resultado: Reprovado.\n");
}

return 0;
}

```

Veja quanta coisa já conseguimos juntar:

```

Array
↓
Guarda várias notas

for
↓
Percorre as notas

Função
↓
Calcula a média

if / else
↓
Verifica o resultado

```

Isso é programação.

Não é decorar 50 comandos.

É aprender pequenas ferramentas e depois aprender a combiná-las para resolver problemas.

Desafio da aula

Agora quero que você tente melhorar o nosso sistema.

Faça o programa receber:



Nome do aluno
5 notas

Depois mostre:

```
=== RESULTADO ===  
  
Aluno: Jose  
Nota 1: 12  
Nota 2: 15  
Nota 3: 10  
Nota 4: 14  
Nota 5: 16  
  
Media: 13.40  
Resultado: Aprovado
```

E tente criar uma segunda função chamada:

```
void mostrarResultado(...)
```

A ideia é começar a separar cada responsabilidade do programa em uma função diferente.

Não precisa conseguir fazer tudo de primeira.

Tente.

Se aparecer um erro, leia a mensagem, descubra onde está o problema e tente corrigir.

É justamente nesse processo que começamos a aprender programação de verdade.

O que aprendemos nesta aula

Nesta aula aprendemos:

- O que são arrays;
- Como criar arrays;
- Como acessar posições de um array;
- Por que a contagem começa em `0`;
- Como percorrer arrays usando `for`;
- Como guardar vários valores em um único array;
- O que são funções;
- Como criar uma função;
- Como chamar uma função;
- Como passar informações para uma função;
- O que são parâmetros e argumentos;

- Como retornar valores usando `return ;`
- A diferença básica entre `void` e `int ;`
- Como combinar arrays, funções, loops e condicionais;
- Como organizar melhor um programa.

Na próxima etapa, podemos começar a trabalhar com **strings, caracteres e manipulação de texto**, e depois avançar para conceitos que vão permitir criar projetos cada vez mais completos.

